



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 7

NOMBRE COMPLETO: Ricardo Emmanuel Galicia Tequianes

N° de Cuenta: 118001740

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 11 de octubre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Práctica 7: Iluminación 1

La escena parte del código base de la práctica 7 con el modelo de Phong activado en el fragment shader. Sobre ella están el rover de la práctica 5, que se mueve con el teclado y lleva un faro azul; el avión de la práctica 6, que sube y baja con una luz amarilla en la parte inferior; y una lámpara importada con su luz puntual blanca. También siguen el dado y el holocrón por código del ejercicio de clase, el helicóptero y el agave del profesor. El proyecto se compiló con CMake y se ejecutó en macOS (OpenGL 4.1 Metal, GLSL 4.10).

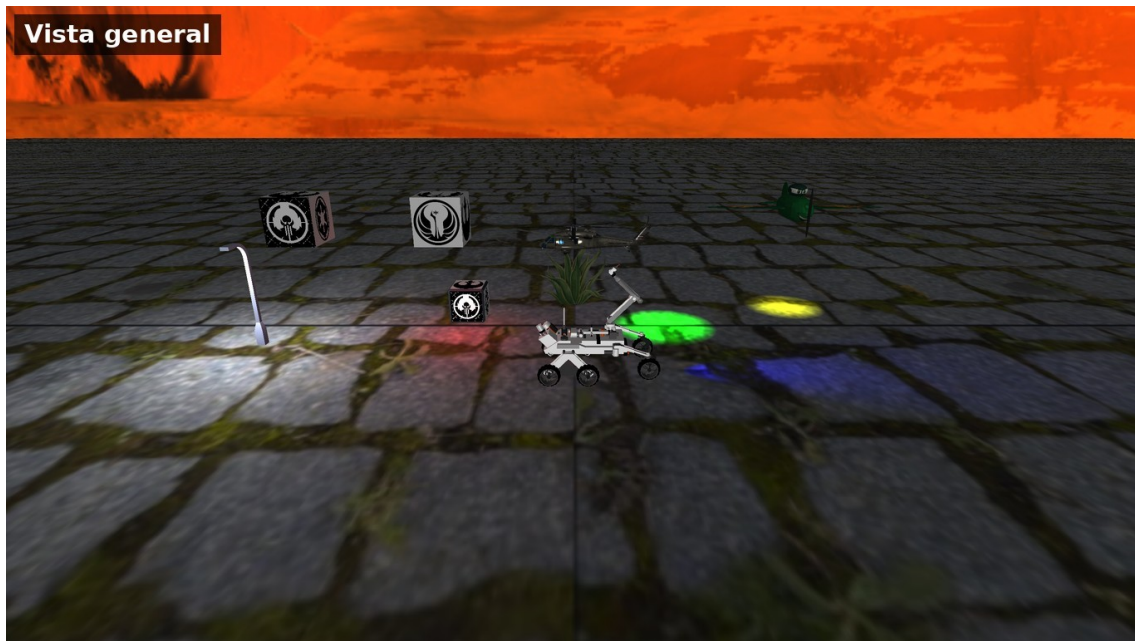


Figura 1. Escena completa con la linterna apagada.

1.- Ejecución de los ejercicios.

a) Movimiento con teclado

El rover avanza con Y y retrocede con U; el avión sube con flecha arriba y baja con flecha abajo. Las teclas se leen en el ciclo principal con el arreglo keys de Window, igual que el control de patas de la práctica 5, y el desplazamiento se multiplica por deltaTime para que no dependa de los cuadros por segundo. Las posiciones se limitan con glm::clamp para que los modelos no salgan del piso. La L enciende y apaga la linterna de la cámara; se compara con el estado del cuadro anterior para que un solo toque cambie el estado una vez.

```

633     bool* keys = mainWindow.getKeys();
634     //Rover: Y hacia adelante (+X, donde está su frente), U hacia atrás
635     if (keys[GLFW_KEY_Y]) posRoverX += VELOCIDAD_ROVER * deltaTime;
636     if (keys[GLFW_KEY_U]) posRoverX -= VELOCIDAD_ROVER * deltaTime;
637     posRoverX = glm::clamp(posRoverX, -15.0f, 15.0f);
638     //Avión: flecha arriba sube, flecha abajo baja
639     if (keys[GLFW_KEY_UP]) posAvionY += VELOCIDAD_AVION * deltaTime;
640     if (keys[GLFW_KEY_DOWN]) posAvionY -= VELOCIDAD_AVION * deltaTime;
641     posAvionY = glm::clamp(posAvionY, 1.0f, 14.0f);
642     //L enciende o apaga la linterna (solo al presionar, no mientras se mantiene)
643     if (keys[GLFW_KEY_L] && !teclaAnterior) linternaEncendida = !linternaEncendida;
644     teclaAnterior = keys[GLFW_KEY_L];

```

Figura 2. Lectura de teclas en el ciclo principal.

Las variables posRoverX y posAvionY entran directamente en la traslación de la matriz base de cada modelo. Esa misma matriz se usa después para colocar su luz.

b) Spotlight amarilla que apunta al piso (avión)

Es spotLights[2]: color (1, 1, 0), intensidad difusa 3, atenuación con 1, lin 0.1 y exp 0.01, y un cono de 25°. Su dirección es fija, (0, -1, 0). La posición se calcula en cada cuadro multiplicando la matriz del avión por el punto (0, -1.05, 0.5), que queda debajo del fuselaje; así la luz sube y baja con el avión sin tener que mover dos cosas por separado.

```

571     //luz del avión: amarilla, en la parte inferior del fuselaje y apuntando al piso;
572     //su posición se actualiza en cada cuadro con la altura del avión
573     spotLights[2] = SpotLight(1.0f, 1.0f, 0.0f,
574         0.0f, 3.0f,
575         0.0f, 0.0f, 0.0f,
576         0.0f, -1.0f, 0.0f,
577         1.0f, 0.1f, 0.01f,
578         25.0f);
579     spotLightCount++;
580
581     //faro del rover: azul, en el frente del cuerpo y apuntando hacia adelante (+X), un poco al piso;
582     //su posición se actualiza en cada cuadro con la posición del rover
583     spotLights[3] = SpotLight(0.0f, 0.0f, 1.0f,
584         0.0f, 5.0f,
585         0.0f, 0.0f, 0.0f,
586         1.0f, -0.2f, 0.0f,
587         1.0f, 0.05f, 0.005f,
588         25.0f);
589     spotLightCount++;

```

Figura 3. Declaración de la spotlight amarilla del avión y del faro azul del rover.

```

672     //Matrices base del rover y del avión: se usan para dibujarlos y para colocar sus luces,
673     //así la luz siempre queda en el mismo punto del modelo aunque este se mueva
674     glm::mat4 baseRover(1.0);
675     baseRover = glm::translate(baseRover, glm::vec3(posRoverX, -1.0f, 6.0f));
676     baseRover = glm::scale(baseRover, glm::vec3(ESCALA_ROVER, ESCALA_ROVER, ESCALA_ROVER));
677     glm::vec3 posFaroRover = glm::vec3(baseRover * glm::vec4(6.9f, 2.3f, 0.0f, 1.0f)); //frente del cuerpo
678     spotLights[3].SetPos(posFaroRover);
679
680     glm::mat4 baseAvion(1.0);
681     baseAvion = glm::translate(baseAvion, glm::vec3(13.0f, posAvionY, -4.0f));
682     baseAvion = glm::scale(baseAvion, glm::vec3(0.75f, 0.75f, 0.75f));
683     glm::vec3 posLuzAvion = glm::vec3(baseAvion * glm::vec4(0.0f, -1.05f, 0.5f, 1.0f)); //debajo del fuselaje
684     spotLights[2].SetPos(posLuzAvion);
685
686     //información al shader de fuentes de iluminación
687     shaderList[0].SetDirectionalLight(&mainLight);
688     shaderList[0].SetPointLights(pointLights, pointLightCount);
689     //si la linterna está apagada se envía el arreglo a partir de la segunda luz
690     if (!linternaEncendida)
691         shaderList[0].SetSpotLights(spotLights, spotLightCount);
692     else
693         shaderList[0].SetSpotLights(spotLights + 1, spotLightCount - 1);

```

Figura 4. Las dos luces se colocan con la matriz de su modelo antes de mandarlas al shader.

Con el avión arriba el círculo es más grande y más tenue: el cono abarca más piso y la atenuación crece con la distancia. Con el avión abajo el círculo es pequeño y brillante.



Figura 5. La luz amarilla con el avión a dos alturas distintas.



Figura 6. Altura inicial del avión y vista cercana.

c) Spotlight azul delantera (faro del rover)

Es spotLights[3]: color (0, 0, 1), intensidad difusa 5, atenuación con 1, lin 0.05 y exp 0.005, y un cono de 25°. Se coloca en el punto (6.9, 2.3, 0) de la matriz del rover, que está en el frente del cuerpo, y apunta en la dirección (1, -0.2, 0): hacia el frente del rover y un poco hacia el piso, para que el círculo quede delante de las llantas.

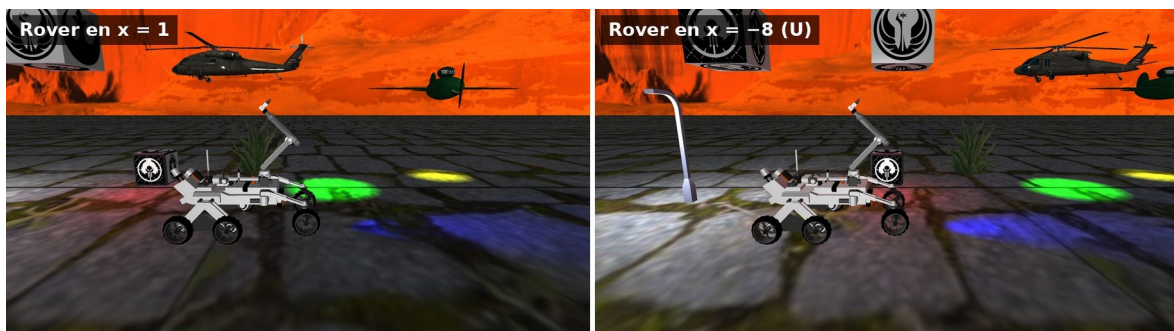


Figura 7. El faro azul se queda delante del rover en las dos posiciones.

Para que el rover tenga jerarquía de llantas separé en Blender cada llanta de su pata. Las llantas tienen su propio material, así que se separaron por material; a cada llanta se le puso el origen en el centro de su caja envolvente y

se exportó centrada, mientras la pata se exportó con sus coordenadas originales. El script guarda también los centros de las llantas, que son los que usa el programa, y el archivo rover_llantas_separadas.blend con la jerarquía cuerpo → pata → llanta.

```

38     for p in piezas:
39         pata = importar("PataRueda_" + p)
40         seleccionar(pata)
41         bpy.ops.object.mode_set(mode='EDIT')
42         bpy.ops.mesh.select_all(action='SELECT')
43         bpy.ops.mesh.separate(type='MATERIAL')
44         bpy.ops.object.mode_set(mode='OBJECT')
45         partes = [o for o in bpy.context.selected_objects]
46         llanta = next(o for o in partes if any("wheels" in m.name for m in o.data.materials))
47         resto = [o for o in partes if o is not llanta]
48         seleccionar(resto[0])
49         for o in resto: o.select_set(True)
50         bpy.ops.object.join()
51         pata = bpy.context.view_layer.objects.active
52         pata.name = "Pata_" + p
53         llanta.name = "Llanta_" + p
54         seleccionar(llanta)
55         bpy.ops.object.origin_set(type='ORIGIN_GEOMETRY', center='BOUNDS')
56         c = llanta.location.copy()
57         # mismo punto en ejes del OBJ (Y arriba): (x, z, -y)
58         info.write(f"{p} {c.x:.4f} {c.z:.4f} {-c.y:.4f}\n")
59         exportar(pata, "Pata_" + p)
60         llanta.location = (0, 0, 0)
61         exportar(llanta, "Llanta_" + p)
62         llanta.location = c
63         # jerarquia en el .blend (despues de exportar y sin mover nada): Cuerpo -> Pata -> Llanta
64         for hijo, padre in ((pata, cuerpo), (llanta, pata)):
65             mundo = hijo.matrix_world.copy()
66             hijo.parent = padre
67             hijo.matrix_world = mundo

```

Figura 8. Parte del script de Blender que separa la llanta, le pone el origen en su centro y exporta las dos piezas.

```

721 //Rover de la práctica 5 con jerarquía: cuerpo -> pata -> llanta
722 //Giro de las llantas = distancia recorrida / radio; negativo porque avanza hacia +X
723 //y la llanta gira sobre su eje Z
724 float giroLlantas = -(posRoverX - INICIO_ROVER_X) / RADIO_LLANTA;
725 model = baseRover;
726 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
727 Material opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
728 Cuerpo.M.RenderModel(uniformColor);
729 Brazo.M.RenderModel(uniformColor);
730 for (int i = 0; i < 6; i++)
731 {
732     //pata: gira sobre su punto de unión con el cuerpo (en la práctica 5 se movía con teclado)
733     modelaux = model;
734     modelaux = glm::translate(modelaux, hingePata[i]);
735     modelaux = glm::rotate(modelaux, 0.0f, glm::vec3(0.0f, 0.0f, 1.0f));
736     modelaux = glm::translate(modelaux, -hingePata[i]);
737     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
738     Pata_M[i].RenderModel(uniformColor);
739     //llanta: hija de la pata, se lleva a su centro y ahí gira
740     modelaux = glm::translate(modelaux, centroLlanta[i]);
741     modelaux = glm::rotate(modelaux, giroLlantas, glm::vec3(0.0f, 0.0f, 1.0f));
742     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
743     Llanta_M[i].RenderModel(uniformColor);
744 }

```

Figura 9. Dibujo del rover: cada llanta toma la matriz de su pata, se traslada a su centro y gira sobre Z.

El giro de las llantas se calcula con la distancia recorrida desde la posición inicial entre el radio de la llanta (1.241 por la escala de 0.4). El signo es negativo porque el rover avanza hacia +X y la llanta gira sobre su eje Z.

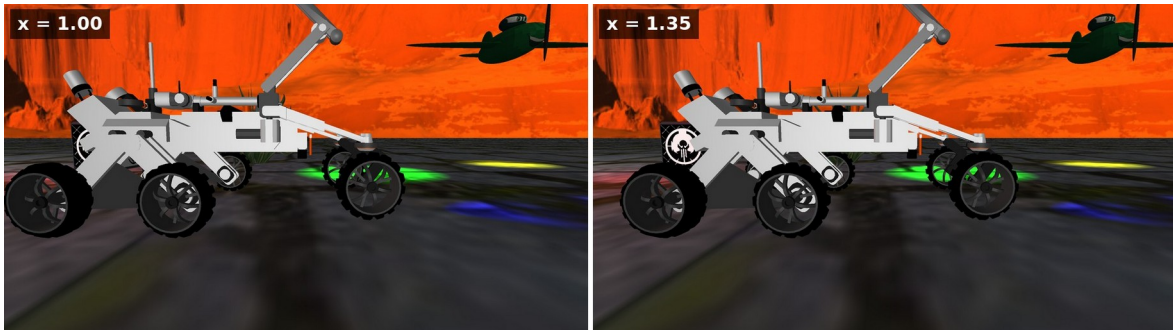


Figura 10. La misma llanta antes y después de avanzar 0.35 unidades.

d) Lámpara importada con luz puntual blanca

La lámpara es el modelo light-curved del paquete City Kit (Roads) de Kenney, que tiene licencia CC0. Se carga con la clase Model; su textura se llama colormap.png, así que la renombré a colormap_ciudad.png y cambié la referencia en el .mtl para que no se confundiera con otra textura del mismo nombre. Está en $(-16, -1, 2.6)$, escalada 8 veces y girada 180° para que el brazo quede hacia el carril del rover.

La luz es pointLights[1]: blanca $(1, 1, 1)$, intensidad difusa 2, atenuación con 1, lin 0.09 y exp 0.02. Su posición sale de multiplicar la matriz de la lámpara por el punto $(0, 0.62, -0.175)$, justo debajo de la pantalla; si después se mueve o se escala la lámpara, la luz se queda en el mismo lugar del modelo.

```

539 //Lámpara: se para junto al carril del rover, girada 180° para que su brazo quede sobre el carril.
540 //La luz puntual blanca va justo debajo de la pantalla de la lámpara.
541 glm::mat4 modeloLampara(1.0);
542 modeloLampara = glm::translate(modeloLampara, glm::vec3(-16.0f, -1.0f, 2.6f));
543 modeloLampara = glm::rotate(modeloLampara, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
544 modeloLampara = glm::scale(modeloLampara, glm::vec3(8.0f, 8.0f, 8.0f));
545 glm::vec3 posFocoLampara = glm::vec3(modeloLampara * glm::vec4(0.0f, 0.62f, -0.175f, 1.0f));
546 pointLights[1] = PointLight(1.0f, 1.0f, 1.0f,
547     0.0f, 2.0f,
548     posFocoLampara.x, posFocoLampara.y, posFocoLampara.z,
549     1.0f, 0.09f, 0.02f);
550 pointLightCount++;

```

Figura 11. Matriz de la lámpara y su luz puntual blanca.



Figura 12. El piso alrededor de la lámpara recibe la luz blanca mientras el rover pasa junto a ella.

Controles: W, A, S, D y ratón mueven la cámara; Y y U, el rover; flecha arriba y flecha abajo, el avión; L enciende o apaga la linterna; Esc cierra el programa.

2.- Problemas y cómo se resolvieron.

Escena sin iluminación en macOS. `glValidateProgram` fallaba con «No vertex array object bound» y `Shader_light.cpp` hacía `return` antes de pedir las ubicaciones de los uniform, así que ninguna luz llegaba al shader. Dejé el mensaje como aviso y quité el `return`.

```

74 //En macOS la validación falla si todavía no hay un VAO enlazado ("No vertex array object bound").
75 //No es un error del shader, así que solo se informa y se continúa para obtener las ubicaciones uniform.
76 glValidateProgram(shaderID);
77 glGetProgramiv(shaderID, GL_VALIDATE_STATUS, &result);
78 if (!result)
79 {
80     glGetProgramInfoLog(shaderID, sizeof(eLog), NULL, eLog);
81     printf("Aviso al validar el programa: '%s'\n", eLog);
82 }

```

Figura 13. La validación ya no detiene la carga de los uniform.

Compilación en la Mac. Los `#include` con diagonal invertida (`gtc\matrix_transform.hpp`) no compilan con clang y se cambiaron por diagonal normal. `practica7.cpp` incluía `Sphere.h`, que no venía en la carpeta y no se usa, así que quité ese include. También agregué un `CMakeLists.txt` que copia shaders, Models y Textures junto al ejecutable.

Contexto de OpenGL. `Window.cpp` pedía OpenGL 4.3 y macOS solo da hasta 4.1 en perfil Core. Se pidió 3.3, que es lo que usan los shaders (`#version 330`).

Cámara girando sola al iniciar. El constructor `Window(ancho, alto)` no inicializaba `mouseFirstMoved`, `lastX` ni `lastY`. Los inicialicé.

Normales. En CreateObjects, calcAverageNormals se llamaba después de crear las mallas y la pirámide y el agave llegaban a la GPU con normales en cero; moví las llamadas antes. Mi dado de la práctica 6 tenía las normales hacia afuera, y con este shader cada cara se iluminaba solo cuando la luz estaba detrás; se invirtió el signo en AgregarCara y en CrearHolocron.

Patas giradas al exportar desde Blender. El importador de OBJ deja una rotación de 90° en cada objeto. Al emparentar las patas con el cuerpo antes de exportarlas, heredaban esa rotación y el OBJ salía girado sobre X. Exporté primero cada pieza y armé la jerarquía después, conservando su posición.

Textura faltante del rover. El material del brazo apunta a SpacialRoverT301Texture.png, que no está. LoadTextureA ahora regresa false cuando no encuentra la imagen y Model usa plain.png en su lugar, en vez de dejar una textura vacía.

3.- Conclusión:

- a. Los ejercicios del reporte: complejidad, explicación.

La complejidad fue media. Las dos spotlight se resuelven igual: la posición de la luz se calcula con la matriz que se usa para dibujar el modelo, así que el faro y la luz del avión no se separan aunque cambien la posición o la escala. La luz puntual de la lámpara es fija y solo hubo que elegir el punto debajo de la pantalla. Lo que más trabajo dio fue la jerarquía de llantas del rover, porque hubo que separar las piezas en Blender sin mover la bisagra de cada pata y calcular el giro a partir de la distancia recorrida.

- b. Comentarios generales.

Faltó explicar en clase hacia dónde deben ir las normales con este shader; se entiende revisando CalcLightByDirection y viendo que el piso usa (0, -1, 0). Sugiero que el código base traiga la línea de iluminación del fragment shader ya activa y con un comentario sobre esa convención.

- c. Conclusión.

Con las tres fuentes del modelo de Phong se ve qué controla cada parámetro: el color y la intensidad difusa definen cuánto ilumina la luz, la atenuación (con, lin, exp) define qué tan rápido se apaga con la distancia y el ángulo de la spotlight define el tamaño del círculo. Poner las luces en la jerarquía de los modelos fue lo que permitió que se movieran con ellos sin código extra para cada tecla.

1. Bibliografía en formato APA

- Blender Foundation. (s. f.). *Wavefront OBJ*. Blender Manual.
https://docs.blender.org/manual/en/latest/files/import_export/obj.html
- de Vries, J. (s. f.). *Basic lighting*. LearnOpenGL.
<https://learnopengl.com/Lighting/Basic-Lighting>
- de Vries, J. (s. f.). *Light casters*. LearnOpenGL.
<https://learnopengl.com/Lighting/Light-casters>
- Kenney. (2026). *City Kit (Roads)* [Modelos 3D, licencia CC0].
<https://kenney.nl/assets/city-kit-roads>
- Roque RG, J. (2026). *Manual de prácticas: Práctica 7. Iluminación 1* [Material de clase]. Laboratorio de Computación Gráfica e Interacción Humano Computadora, Facultad de Ingeniería, UNAM.
- The Khronos Group. (s. f.). *glValidateProgram*. OpenGL 4 Reference Pages.
<https://registry.khronos.org/OpenGL-Refpages/gl4/html/glValidateProgram.xhtml>